DUDOXX ENGINEERING RECORD

icedLiquid

Building a voice assistant around one model in one browser tab

Built on LFM models by Liquid AI (https://www.liquid.ai).

WebGPU | LFM3-Omni | ASR · tools · TTS · turn model | Every number from the register

Author
Walid Boudabbous — Dudoxx UG / Acceleate Consulting

Session run
20261030T024341Z

Date
2026-10-30

Liquid AI's LFM3-Omni takes text, images, video and audio in, and gives text and speech out. We wanted to know how far that one model can carry a spoken assistant when it runs in a Chrome tab on WebGPU, with no inference server behind it. The result is icedLiquid. This article is the engineering record of the work on one Mac: what we built, in which order, what broke, which decisions we took, and what is still open.

The constraint shaped every choice. One model does the listening, the thinking and the speaking. One GPU worker runs it, one job at a time. A small model will not always follow instructions, so the code around it has to catch what the model misses. And every number we publish comes from a measurement register that points back to a raw run file.

Where each part runs

The tab holds the model. A Web Worker loads the LFM3-Omni ONNX graphs, keeps the weights in the browser's private file system, and runs the brain, the tool rounds and the speech synthesis. The microphone, the voice activity detector (VAD), the half-duplex gate, barge-in and the echo guard also live in the page.

The server holds the keys. The Next.js app serves the page and the model files, and it hosts the tool routes for weather, time and web search. The browser calls those routes, so no API key ever reaches the client. Behind the app, a local gateway fronts two engines on the Mac: MLX-VLM in 4-bit first, llama.cpp as the fallback. The gateway serves the server-side speech recognition lane and every benchmark in this article.

The ears can use either side. A config switch, `voice_agent.asr_lane`, selects the gateway lane or the browser lane. Today it is set to `browser`, a design decision of 2026-10-30. How we got there is the longest part of the story.

The starting point: which engine serves the model

Before any assistant work, we probed what the model can do on this Mac. The probe matrix covered 9 input and output cells, from plain text to video with sound and speech out. Only MLX served all of them. llama.cpp has no video path and answers it with a loud refusal; vLLM was excluded because the model card does not list it and the family has no decoder for the speech codec.

The probe also caught two traps early. The MLX processor silently ignores the model card's flags for reading the audio track of a video, so video with sound works only when we extract the soundtrack and pass it as a separate audio input. And an image smoke test passed on a blank image, because the model described text that was not there. We regraded on a real frame. Both lessons fed one rule for the rest of the project: a test that accepts any fluent answer proves nothing.

On short prompts, MLX decoded at 187.9 tok/s on this machine, the server ceiling for everything that follows.

The ears: realtime speech recognition, lane by lane

THE MODEL HAS NO STREAMING ENCODER

A voice assistant needs words while the user is still talking. File transcription on MLX worked from the start, with a median latency of 102.0 ms per clip on our test set. The question was whether the model could stream.

We tested llama.cpp's realtime endpoint with a probe that owned its own server and timestamped every message on the wire. The rule was strict: a verdict needs a transcript delta before the audio commit on every clip, and an empty or failed clip raises an error instead of a verdict. The answer was clear. Clip one produced 0 deltas before the commit; text arrived only after it. The verdict was batch only. The reason is in the model's chat template: live recognition needs an `asr` template, which the separate LFM3-Audio live checkpoints carry and Omni does not.

The probe's first verdict was also wrong, and we kept the record. Its first run opened a conversation session instead of a transcription session. No text came back, but the rule still printed a verdict. We fixed the session type, made the rule raise on empty data, re-ran, and marked the first run as superseded.

PSEUDO-STREAMING ON THE MLX LANE

With a batch encoder, we built streaming around it. The page runs an energy VAD. While speech continues, it re-sends the growing utterance to the gateway every 500 ms. A LocalAgreement rule marks a word as stable once 2 consecutive passes agree on it. A silence of 700 ms ends the utterance and produces the final text.

On the live bench this gave a first interim at 455 ms (p50) and a final at 211 ms (p50) after the pause. The word error rate was 0.0 %, with 0 retractions of stable words. The cost is GPU duty: the GPU worked for a share of 0.25 of the speech time instead of one request per clip. We chose a plain POST of a WAV file to the existing relay over WebSocket or server-sent events, because a Next.js route handler cannot upgrade to a socket and a custom server would break the standalone Docker build.

Small dead ends piled up here. The sentence splitter did not split "nine. please", so we added a rule for a lowercase word after a full stop. Vocabulary hints helped names, but on noise-only audio the model echoed the hint list back, so an echo guard drops that output.

Source: the `realtime.*`, `asr_live.*` and `mlx.asr.*` rows of the register.

THE BROWSER LANE FAILED, THEN PASSED A REPLAY

The assistant was meant to run entirely in the tab, so we wired the same VAD to the model's `audio_to_text` mode on WebGPU. On a fixture file this lane produced its first token after 2059 ms, slow enough that we switched interim passes off for it.

Then the live microphone check failed. In 7 of 7 passes the model returned zero tokens on microphone audio, so it produced text in 0 of 7. We moved the ears to the gateway lane and opened a diagnosis plan. It built a capture store for real microphone audio, an outside judge, and worker diagnostics for loudness, mel floor and the end-of-sequence probability of the first token. We wrote down hypotheses: aliasing from a nearest-neighbour downsample with no low-pass filter, the browser's gain control and noise suppression, buffer drops in the audio processor. We refuted a sample-rate mismatch and stereo input.

A replay of one real microphone capture through every lane then passed. The browser lane returned text on a non-empty share of 1.00, and the gateway lane scored a word error rate of 5.7 % at a median request time of 166.0 ms. On that evidence we switched the default to the browser lane. The register still marks the browser row as an open gap, and we agree. The root cause of the first failure was never written down, the fixes from the diagnosis are unfinished, and a replay is not a person speaking.

On 2026-10-30 we ran a spoken test through the air: a scripted voice played from the laptop speakers, the laptop microphone picked it up, and the page listened. The browser lane transcribed every scripted line correctly, in English and in German, in a quiet room. That run has no register rows yet, so we cite its evidence file and give no figures. It is the best sign so far that the browser lane works on real acoustic input, and it is still one run.

Source: the `voice.browser_asr.*` and `mic_asr.*` rows, and the spoken-capture note in the Sources page.

The brain and the tool agent

FROM HAND-WRITTEN TOOLS TO A REAL TOOL INTERFACE

The first tool test put the tool definitions into the system turn by hand and scored 16/20 calls, or 80.0 %. The misses were instructive: a number written in digits where the case expected words, a duration in the wrong unit, a missing call. We then gave the gateway an OpenAI-style `tools` field, so the engine renders the tools in the model's own template and parses its own call syntax.

A dedicated bench followed: a pool of 10 tools, 20 cases including some that want no call, and 550 graded calls per engine. MLX picked the right call in 100.0 % of the cases at N=100. That grading is lenient. It matches arguments by content, not against a strict schema, and every case is a short English single call. Multi-step, parallel and non-English calls are not measured.

Source: the `tools.*` rows of the register.

THE LOOP IN THE TAB

In the browser, the brain runs the WebGPU `tools` mode. The model reads the conversation, may emit a call, the page runs it against the server route, and the result goes back for a spoken answer. The loop allows at most 2 tool rounds per turn and keeps the last 6 turns of

history. Tools are locale-aware: a German place name such as "Tokio" resolves through a geocoder in that language. A tool with a missing key returns a failure object, never invented data.

In single live calls, the weather tool answered in 212 ms, the time tool in 128 ms and web search in 3170 ms. These are single calls from an earlier run, not a benchmark.

PHRASE LISTS WHERE THE MODEL WOULD NOT CALL A TOOL

We added page tools for the theme, the language and the end of the session. The small model mostly answered in words instead of calling them: it said it had switched to dark mode and did nothing. We stopped relying on it. Deterministic phrase lists in the config, one per supported language, now drive the theme switch, the goodbye and the language switch. A language switch needs a switch verb plus exactly one language name, so "what is that called in French?" does not move the page.

THE GROUNDING GUARD, AND WHERE IT STILL FAILS

The model invents numbers. In one early session it reported a temperature for a city without calling the weather tool. The first saved session had the same fault in another form: asked for the time in Tokyo, it answered "seven thirty" with no tool call, and the answer was wrong.

So we added a grounding guard. An answer that carries a data number, a digit next to a unit or a clock time, is never spoken when no tool ran in that turn. The brain gets one retry with a nudge, then the page speaks a fixed "I could not check that" line in the session language.

The spoken test showed that the guard is not enough. The Tokyo question again got a wrong time with no tool call. A German question about the weather in Berlin got an English answer, with no tool call, that reused the Hamburg figures from an earlier turn. Our reading, not yet tested: the system prompt tells the model to write numbers as spoken words for the voice, and a guard that looks for digits cannot see "seven thirty". The Berlin answer also shows that the history window gives the model old tool results it can copy. Grounding has to move from a text pattern to a rule about which turns may state current facts at all.

The voice: speech out of the same model

On the server, the model speaks through its own speech codec. Our first attempt to answer a spoken question in pure audio mode produced unrelated speech. The working form is the voice-chat mode with interleaved text and audio, set by its own system prompt. On MLX, the first audio of that mode arrived at 262.0 ms (p50).

In the browser we had to port the speech path to TypeScript: the depth decoder that samples the codec codes over several passes, the codec decode, and an inverse short-time Fourier transform with a mixed-radix FFT. Each step was checked against a Python reference. The port caught a range error in our own spec: valid codec codes stop one value earlier than the language-model side suggests, and end-of-sequence frames never reach the codec. The browser output is not bit-equal to MLX, because the random streams differ.

A standalone text-to-speech pass in the browser reached its first audio after 3259 ms. That is a single, cold pass and it measures something else than the in-session figure below. In the assistant, the voice streams: speech frames are flushed in small batches and played as they arrive, so the user hears the start while the rest is still being generated.

Two details made the voice feel right. The screen shows the answer as Markdown, but the voice reads a cleaned speech text without marks or links. And the animated orb first followed the energy of generated chunks, which ran ahead of the audible speech; it now follows a playback analyser on the real output.

Localization: one prompt set, four languages

icedLiquid speaks English, German, French and Spanish, and the session language is the page locale. Every model-facing string is a template in the config: the system prompt, the tool follow-up, the speech-recognition and speech-synthesis turns, the greeting and the "could not check" line. No English-only turn is left in the code.

File transcription in German, French and Spanish worked word for word on synthetic clips, with latencies of 144 ms, 144 ms and 165 ms. Those clips were read by a person, not scored by a judge. The model must answer in the session language even when a tool result is in another one, and a spoken language switch moves the page to the new locale and resumes the session.

The limits are plain. Only the English voice has a published quality score. German, French and Spanish speech quality is not measured, and neither is end-to-end quality of the assistant in those languages. The Berlin failure above is a localization failure as much as a grounding one.

The WebGPU engine: one worker, one cache, hard memory limits

The browser model is the public ONNX export of LFM3-Omni. The text decoder ships in `q4f16`; the vision, audio, depth and codec side graphs follow the model card's precisions, because no 4-bit versions exist. One worker loads the graphs, reads the cache layout from the session metadata, and disposes GPU tensors every step. The tab passed all modes in one live run, 10 passed, with every parser and signal step checked against Python references.

Three failures shaped the engine. First, the browser's Cache Storage refused to store the decoder's data file, which is larger than a gigabyte. We moved the weights to the Origin Private File System, keyed by the server's ETag and the byte size. Second, a strict "no CPU nodes" setting failed the speech graphs, because ONNX Runtime places shape operations on the CPU by design; we changed that rule to a warning. Third, a renderer crash cannot be caught in JavaScript. A long 1080p video killed the tab with no error, so the page now estimates the decode size before it reads any frame and refuses above 1 GiB.

Memory is the binding limit. The renderer peaked at 2916 MB while the decoder loaded, and the GPU process reached 4756 MB across all modes, above our own budget. An upload of 272 MB crashed the tab, and there is still no upload cap.

On short prompts the browser produced its first token in 51.8 ms (p50) and decoded at 43.5 tok/s. A cold load of the model took 6015.3 ms in the bench run; a load from the private file system took 1477.3 ms in the recorded session. The two come from different runs, so read them as an order of magnitude.

The voice loop: half duplex, echo and barge-in

The loop is half duplex by design. One model on one worker does one job at a time, so the microphone gate closes while the assistant thinks and speaks. That choice created its own bug. When the microphone reopened at the end of playback, it heard the assistant's last words through the room and answered them, in a loop of "Thank you." and "Okay.". The fix is an echo tail: the microphone stays shut for 600 ms after playback ends.

Room noise was the next problem. The shared VAD let clicks and breaths through, and the recogniser turned some of them into phantom subtitle credits in French. The assistant got its own louder and longer VAD, a list of phantom phrases, and a filler rule: a lone "uh" or "yeah" shortly after the assistant spoke is dropped as echo or noise.

Barge-in lets the user cut the assistant off. The page compares the microphone level with the playback level, and it stops generation and playback when the user is clearly louder for 300 ms. The threshold is relative to playback so the assistant's own voice in the microphone does not trigger it. Barge-in is built, but no recorded session has exercised it, so we do not claim it works.

Deciding when a turn is finished

The first saved session showed the gap clearly. The room was asked to be quiet, yet the open microphone produced short unscripted turns: "Yeah.", "Thank you.", "Uh.", "So.". Each became a real agent turn. A VAD knows when sound stops; it does not know whether the speaker has finished a thought.

We looked for a turn model first. Liquid AI publishes decision models, but none is named for turn-taking. We compared audio-based detectors, a text-based detector that its own documentation marks as deprecated, and Laya, the turn model already used in our production assistant. Laya is a multilingual encoder with a small head. It reads the transcript and scores one question: will the speaker continue, or is the turn complete or incomplete?

The proof of concept ran on 2026-10-30. We exported the model to ONNX and compared it with the PyTorch original on a replay set in English, German, French and Spanish; the full-precision graph matched exactly. Naive dynamic int8 quantisation failed twice: first with a shape-inference error that needed a workaround, then with predictions that moved far from the original, and with operators WebGPU does not support. Among the other variants, only half precision with layer norms kept in full precision met the parity bar. We chose that graph. The noise probe showed why the bar matters: under int8, the decision on "So." flipped.

At the time of these measurements the gate was not wired. It stayed off in the config, `enabled: false`, until a browser bench passed. WebGPU parity, browser latency, the extra GPU memory, and whether the gate actually removes "Uh." and "So." in a live session are all still open. The Laya results have no register rows, so this section carries no figures; the evidence files are on the Sources page.

What we measured in a live session

On 2026-10-30 we saved one session with per-turn timings (run `20261030T024341Z`). It has 12 answered turns. The scripted questions were typed, because the tester was not at the microphone, and the noise turns described above are in the same aggregates. The ears figure comes from the spoken turns only.

TIMING	MS
Reply, end of question to first audio, p50	1121.1
Reply, end of question to first audio, p90	2976.0
Ears, final recognition pass, p50	161.1
Brain thinking time, p50	837.9
Speak start to first audio, p50	282.9

GENERATION RATE	TOK/S
Brain, p50	14.4
Voice codec tokens, p50	24.0

The median reply arrived in about a second. The p90 reply was the weather turn, which needs a second model round after the tool call. The slowest turn was the web search; it sits above the p90 and has no register row of its own. The context grew from 1150 tokens on the first turn to 1491 on the last.

What is still open

- **Grounding.** The guard missed a wrong time and a wrong-city, wrong-language weather answer in the spoken test. It needs a rule that does not depend on digits.
- **Browser speech recognition.** The first live failure has no recorded root cause. The replay bench and one spoken run passed; neither is a broad human test.
- **Turn decisions.** Noise and fillers still become turns until the Laya gate is wired and measured in the browser.
- **Barge-in.** Built, not exercised in a recorded session.
- **Start button.** At capture time, a first press on a cold page only loaded the model. A later change fixed one-press start, but the spoken run saw the old behaviour once more, so it is intermittent.

- **Memory.** The GPU footprint is above budget, graphs are never evicted, and uploads have no size cap.
- **Languages.** German, French and Spanish speech quality and end-to-end quality are not measured.
- **Scope.** One Mac, one browser profile, one model build. These are not fleet numbers.

How to reproduce

1. Clone https://github.com/Dudoxx/icedliquid and provision the browser weights once with `pnpm models:provision`. They land in a git-ignored local model folder.
2. Start the web app with `pnpm dev` and open `http://127.0.0.1:3400/en/voice-agent`; the port, 3400, comes from the engine configuration.
3. Press "Start talking", then ask for the weather, the time and a web search, and press "Save run".
4. The saved `session.json` holds the per-turn timings. Our internal judge script turned it into the aggregates above.
5. Every number in this article was checked by a script against our internal measurement register before printing. The Sources page names the register row, the date and the run of each figure.

Thank you, Liquid AI

This project exists because the team at Liquid AI released LFM3-Omni as one open model that can listen, think and speak, with an ONNX export small enough to run in a browser tab. Their model card, samples and chat templates answered most of our questions, and the parts we got wrong were ours. Thank you for the models and the documentation. Learn more at https://www.liquid.ai.

Our next steps are the grounding rule, the Laya turn gate in the browser, and a human test of the browser microphone lane. When those rows move from open to measured, we will publish the runs.

Sources

Every figure in this article resolves to one row of the Dudoxx lab measurement register or one key of the engine configuration. A register value stored with more than three decimals is printed rounded to one decimal; the register keeps the full value. The logs themselves are internal.

REGISTER ROW OR CONFIG KEY	SHOWN	UNIT	PROVENANCE
`asr_live.agreement`	2		design setting in the engine configuration
`asr_live.endpoint_silence_ms`	700		design setting in the engine configuration
`asr_live.interim_interval_ms`	500		design setting in the engine configuration
`ui.port`	3400		design setting in the engine configuration
`voice_agent.asr_lane`	browser		design setting in the engine configuration
`voice_agent.barge_in.hold_ms`	300		design setting in the engine configuration
`voice_agent.echo_tail_ms`	600		design setting in the engine configuration
`voice_agent.history_turns`	6		design setting in the engine configuration
`voice_agent.max_tool_rounds`	2		design setting in the engine configuration
`voice_agent.name`	icedLiquid		design setting in the engine configuration
`asr_live.single.final_p50`	211	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T194742Z (internal benchmark log)
`asr_live.single.first_interim_p50`	455	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T194742Z (internal benchmark log)
`asr_live.single.gpu_duty`	0.25		Measured in the Dudoxx lab, 2026-10-29, run 20261029T194742Z (internal benchmark log)
`asr_live.single.retractions`	0		Measured in the Dudoxx lab, 2026-10-29, run 20261029T194742Z (internal benchmark log)
`asr_live.single.wer`	0.0	%	Measured in the Dudoxx lab, 2026-10-29, run 20261029T194742Z (internal benchmark log)
`asr_ml.de.latency`	144	ms	Measured in the Dudoxx lab, 2026-10-29 (internal session evidence log)
`asr_ml.es.latency`	165	ms	Measured in the Dudoxx lab, 2026-10-29 (internal session evidence log)
`asr_ml.fr.latency`	144	ms	Measured in the Dudoxx lab, 2026-10-29 (internal session evidence log)

REGISTER ROW OR CONFIG KEY	SHOWN	UNIT	PROVENANCE
`mic_asr.browser.ok_share`	1.00		Measured in the Dudoxx lab, 2026-10-30, run 20261030T000218Z (internal session evidence log)
`mic_asr.gateway.request_ms_p50`	166.0	ms	Measured in the Dudoxx lab, 2026-10-30, run 20261030T000218Z (internal session evidence log)
`mic_asr.gateway.wer_pct`	5.7	%	Measured in the Dudoxx lab, 2026-10-30, run 20261030T000218Z (internal session evidence log)
`mlx.asr.latency_median`	102.0	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T043449Z (internal probe log)
`mlx.bench.c1.ttfa_p50`	262.0	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T024603Z (internal run notes)
`mlx.probe.matrix_size`	9		Measured in the Dudoxx lab, 2026-10-29, run 20261029T020231Z (internal probe log)
`realtime.clip01.deltas_before_commit`	0		Measured in the Dudoxx lab, 2026-10-29, run 20261029T045204Z (internal probe log)
`text.mlx.short.c1.decode_p50`	187.9	tok/s	Measured in the Dudoxx lab, 2026-10-29, run 20261029T051430Z (internal benchmark log)
`tools.cases`	20		Measured in the Dudoxx lab, 2026-10-29, run 20261029T135702Z (internal benchmark log)
`tools.mlx.calls`	550		Measured in the Dudoxx lab, 2026-10-29, run 20261029T135702Z (internal benchmark log)
`tools.mlx.n100.correct_pct`	100.0	%	Measured in the Dudoxx lab, 2026-10-29, run 20261029T135702Z (internal benchmark log)
`tools.pilot.correct`	16/20		Measured in the Dudoxx lab, 2026-10-29 (internal run notes)
`tools.pilot.correct_pct`	80.0	%	Measured in the Dudoxx lab, 2026-10-29 (internal run notes)
`tools.pool_size`	10		Measured in the Dudoxx lab, 2026-10-29, run 20261029T135702Z (internal benchmark log)
`voice.browser_asr.ok`	0 of 7		Measured in the Dudoxx lab, 2026-10-29 (internal run notes)
`voice.browser_asr.passes`	7 of 7		Measured in the Dudoxx lab, 2026-10-29 (internal run notes)
`voice.search.latency`	3170	ms	Measured in the Dudoxx lab, 2026-10-29 (internal run notes)
`voice.time.latency`	128	ms	Measured in the Dudoxx lab, 2026-10-29 (internal run notes)
`voice.weather.latency`	212	ms	Measured in the Dudoxx lab, 2026-10-29 (internal run notes)
`voice_session.brain_ms_p50`	837.9	ms	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)

REGISTER ROW OR CONFIG KEY	SHOWN	UNIT	PROVENANCE
voice_session.brain_tok_s_p50	14.4	tok/s	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.context_tokens_first	1150		Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.context_tokens_last	1491		Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.ears_ms_p50	161.1	ms	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.reply_ms_p50	1121.1	ms	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.reply_ms_p90	2976.0	ms	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.turns	12		Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.voice_first_audio_ms_p50	282.9	ms	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
voice_session.voice_tok_s_p50	24.0	tok/s	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
webgpu.mem.gpu_footprint_peak_10modes	4756	MB	Measured in the Dudoxx lab, 2026-10-29, run 20261029T182135Z (internal session evidence log)
webgpu.mem.renderer_peak_high	2916	MB	Measured in the Dudoxx lab, 2026-10-29, run 20261029T182135Z (internal session evidence log)
webgpu.mode.audio_to_text.ttft	2059	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T182135Z (internal session evidence log)
webgpu.mode.text_to_speech.ttfa	3259	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T182135Z (internal session evidence log)
webgpu.model_load_cache_ms	1477.3	ms	Measured in the Dudoxx lab, 2026-10-30, run 20261030T024341Z (internal session evidence log)
webgpu.model_load_ms	6015.3	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T163833Z (internal benchmark log)
webgpu.modes_passed	10 passed		Measured in the Dudoxx lab, 2026-10-29 (internal session evidence log)
webgpu.short.decode_p50	43.5	tok/s	Measured in the Dudoxx lab, 2026-10-29, run 20261029T163833Z (internal benchmark log)
webgpu.short.ttft_p50	51.8	ms	Measured in the Dudoxx lab, 2026-10-29, run 20261029T163833Z (internal benchmark log)
webgpu.upload_crash	272	MB	Measured in the Dudoxx lab, 2026-10-29, run 20261029T182135Z (internal session evidence log)
webgpu.video_preflight_limit	1	GiB	Measured in the Dudoxx lab, 2026-10-29, run 20261029T182135Z (internal session evidence log)

EVIDENCE WITHOUT REGISTER ROWS

These internal records back the qualitative statements. They carry no register row, so the article quotes no figure from them.

- R&D notes of the Dudoxx lab: chronology, architecture, dead ends, per-topic write-ups (internal).
- Spoken acoustic test, run 20261030T042920Z: browser ASR transcripts, Tokyo and Berlin grounding failures (internal session log).
- First saved session: typed script, unscripted noise turns, ungrounded Tokyo answer (internal session log).
- Laya ONNX proof of concept, run 20261030T031234Z: export, fp32 parity, int8 failure, noise probe (internal result log).
- Laya graph variants and the chosen fp16 graph with fp32 layer norms (internal result log).
- Browser microphone ASR diagnosis: hypotheses and refuted causes (internal notes).

PUBLIC SOURCES

- Liquid AI, LFM models: https://www.liquid.ai/models
- LFM3-Omni-2B ONNX model card (Hugging Face): https://huggingface.co/LiquidAI/LFM3-Omni-2B-ONNX
- ONNX Runtime Web: https://onnxruntime.ai/docs/tutorials/web/
- WebGPU specification (W3C): https://www.w3.org/TR/webgpu/
- icedLiquid source code: https://github.com/Dudoxx/icedliquid